quick_sort.cpp

```cpp
#include <stdio.h>
#include <omp.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        #pragma omp parallel sections
        {
            #pragma omp section
            {
                quickSort(arr, low, pi - 1);
            }
            #pragma omp section
            {
                quickSort(arr, pi + 1, high);
            }
        }
    }
}
int main()
{
    int arr[] = {50, 20, 80, 10, 60, 30, 90, 40, 70};
    int n = 9;
    printf("Original Array:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
    #pragma omp parallel
    {
        #pragma omp single
        {
            quickSort(arr, 0, n - 1);
        }
    }
    printf("\nSorted Array:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}

```